AI の学習と線形代数
―勾配降下法はいつ収束するか―

bv25046 押川龍太

2026 年 10 月 31 日

目次

1 今回の動機 2

2 復習 2
- 2.1 実対称行列の対角化 . . . 3
- 2.2 2 次形式と正定値行列 . . . 5
- 2.3 勾配と 2 次関数 . . . 6

3 AI の学習への利用 7
- 3.1 勾配降下法 . . . 7
- 3.2 誤差の漸化式と対角化 . . . 8
- 3.3 収束の条件 . . . 8
- 3.4 最適な学習率と条件数 . . . 9
- 3.5 例：直線の当てはめ . . . 10
- 3.6 実際の AI のどこで使われているか . . . 11

4 まとめ 12

1　今回の動機

前回（2026 年度大宮祭）は，線形代数で学んだ対角化を復習して，漸化式で決まる数列の一般項を求めるのに使えることを調べました．そのまとめで，線形代数のほかの使われ方も勉強したいと書きました．

今回は AI（機械学習）を取り上げます．画像を見分ける AI や文章を作る AI など，今の AI の多くは「学習」によって作られます．AI の学習とは，予測がどれだけ外れたかを表す関数（**損失関数**）が小さくなるように，AI の中の数（**パラメータ**）を少しずつ動かしていくことです．そのいちばん基本の方法が**勾配降下法**です．

解説

AI の学習は「目隠しをして，山の谷底を目指す」ことにたとえられます．

- 地面の高さ ＝ 損失関数．低いほど，データに対する予測の外れが小さい．
- 自分のいる場所 ＝ パラメータ（AI の中の調整つまみの値）．
- 勾配降下法 ＝ 足元でいちばん急な下り坂の向きに，一歩ずつ進むこと．
- 学習率 ＝ その一歩の大きさ（歩幅）．

谷底（損失がいちばん小さい場所）にたどり着ければ，学習は成功です．

勾配降下法では「**学習率**」という歩幅を自分で決めます．大きすぎると学習が失敗し，小さすぎると時間がかかります．実際には試しながら選ぶことが多いのですが，損失関数が 2 次関数のときは，学習率をどう選べばよいかを線形代数ではっきり答えられます．そこで使うのが，前回も使った固有値と対角化です．この結果は，AI を作るプログラムの「学習率」の設定や，実際の AI の学習で見つかっている現象ともつながっています（3.6 節）．

今回使う主なものは次のとおりです．1〜3 は，前回の資料で挙げた「1 年後期までに学んだもの」の一覧の 6，8，9 にあたります．

1. 固有値，固有ベクトル
2. 内積，ベクトルの長さ
3. 実対称行列の直交行列による対角化
4. 2 次形式と正定値行列（多変量解析で学んだもの）
5. 偏微分（勾配）

特に，前回の「数列への利用」で使った「行列の累乗を対角化で計算する」方法が，今回もそのまま鍵になります．

2　復習

実対称行列の対角化，2 次形式，勾配を扱います．証明も載せていますが，読まなくても分かるように説明していきます．枠で囲んだ「解説」は，初めて読む人向けの説明です．ベクトルはすべて縦ベクトルで，成分は実数とします．

2.1 実対称行列の対角化

前回の資料のとおり，$Av = \lambda v$（$v \neq \mathbf{0}$）となる λ を固有値，v を固有ベクトルといいます．

解説

行列 A をベクトルにかけると，ふつうは向きも長さも変わります．ところが固有ベクトル v は，A をかけても**同じ直線の上にとどまり**，λ 倍されるだけです（λ が負なら逆向きになります）．固有値 λ は「その方向に何倍されるか」を表します．例えば $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ なら $A \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 3 \end{pmatrix}$ なので，$(1,1)^\top$ の方向には 3 倍に伸ばす行列です．

Def 1.1 (転置，対称行列，内積，長さ)**.** 行列 A の行と列を入れ替えた行列を**転置**といい，$A^\top$ と書きます．$A^\top = A$ となる正方行列を**対称行列**といいます．$x, y \in \mathbb{R}^n$ の**内積**を $x^\top y = x_1 y_1 + \cdots + x_n y_n$，**長さ**を $\|x\| = \sqrt{x^\top x}$ とします．

Def 1.2 (直交行列)**.** n 次正方行列 P が $P^\top P = PP^\top = I_n$ を満たすとき，P を**直交行列**といいます．このとき $P^{-1} = P^\top$ で，P の列ベクトルはどれも長さ 1 で，互いに直交しています．

Prop 1.3 (直交行列は長さと内積を変えない)**.** P が直交行列なら，どんな $x, y \in \mathbb{R}^n$ についても $(Px)^\top (Py) = x^\top y$ で，特に $\|Px\| = \|x\|$.

Proof. $(Px)^\top (Py) = x^\top P^\top P y = x^\top y$．$y = x$ とすると $\|Px\|^2 = \|x\|^2$. □

解説

内積が変わらないので，長さも角度も変わりません．直交行列は，平面なら「回転」や「折り返し」のような変換です．直交行列で座標を取り替えることは，方眼紙を回して見直すようなものです．

Thm 1.4 (実対称行列の対角化)**.** A が n 次実対称行列なら，A の固有値はすべて実数で，ある直交行列 $P = (p_1 \ \cdots \ p_n)$ を使って

$$P^\top A P = \begin{pmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_n \end{pmatrix}$$

と対角化できます．ここで p_i は固有値 λ_i の固有ベクトルで，どれも長さ 1 で，互いに直交しています．

Proof. 全体の証明は長いので，参考文献 [1]，[2] を見てください．ここでは「異なる固有値の固有ベクトルは直交する」部分だけ示します．$Ax = \lambda x$，$Ay = \mu y$（$\lambda \neq \mu$）とすると，$A^\top = A$ なので

$$\lambda\, x^\top y = (Ax)^\top y = x^\top A^\top y = x^\top A y = \mu\, x^\top y.$$

よって $(\lambda - \mu) x^\top y = 0$ で，$\lambda \neq \mu$ だから $x^\top y = 0$ です. □

解説

この定理は，「座標軸を固有ベクトル $p_1, \dots, p_n$ の向きに取り替えると，A は**各軸の方向をそれぞれ** λ_i **倍するだけ**の簡単な変換に見える」と言っています．対称行列なら，その新しい座標軸を互いに直交するようにとれるのがポイントです．

Rem 1.5. 前回の対角化 $P^{-1}AP$ と違って，実対称行列なら P を直交行列にとれるので，逆行列は転置 $P^{-1} = P^\top$ で済みます．

ここまでを使って，一つ問題を解いてみます．

Ex 1.6.

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

を直交行列で対角化してください．

SoL. A の固有多項式は

$$\det \begin{pmatrix} t-2 & -1 \\ -1 & t-2 \end{pmatrix} = (t-2)^2 - 1 = (t-1)(t-3)$$

なので，固有値は $1, 3$ です．

固有値 1 の固有ベクトル: $(I_2 - A)v = \mathbf{0}$ は $-x_1 - x_2 = 0$ なので $v = s\begin{pmatrix} 1 \\ -1 \end{pmatrix}$．長さを 1 にして $p_1 = \dfrac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ -1 \end{pmatrix}$．

固有値 3 の固有ベクトル: $(3I_2 - A)v = \mathbf{0}$ は $x_1 - x_2 = 0$ なので $v = s\begin{pmatrix} 1 \\ 1 \end{pmatrix}$．長さを 1 にして $p_2 = \dfrac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ 1 \end{pmatrix}$．

$p_1^\top p_2 = 0$ なので，

$$P := \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix}$$

は直交行列で，$P^\top AP = \begin{pmatrix} 1 & 0 \\ 0 & 3 \end{pmatrix}$ となります．

この対角化から，あとで使う 2 つの性質が出てきます．

Prop 1.7 (固有ベクトルで展開する)**.** *Thm 1.4* の P を使うと，どんな $x \in \mathbb{R}^n$ も

$$x = c_1 p_1 + \cdots + c_n p_n, \qquad c_i = p_i^\top x$$

と書けて，$\|x\|^2 = c_1^2 + \cdots + c_n^2$ です．

Proof. $c = P^\top x$ とおくと $Pc = PP^\top x = x$ で，$Pc = c_1 p_1 + \cdots + c_n p_n$ です．$P^\top$ も直交行列なので，Prop 1.3 から $\|c\| = \|P^\top x\| = \|x\|$. □

解説

$c_1, \ldots, c_n$ は「x を，固有ベクトルの向きの座標で測った値」です．座標を回しただけなので長さは変わらず，三平方の定理と同じ形の式になります．

Prop 1.8 (行列の累乗)**.** $D = P^\top AP$ とすると $A^k = PD^kP^\top$ です．さらに，実数 η について

$$(I_n - \eta A)^k = P \begin{pmatrix} (1-\eta\lambda_1)^k & & \\ & \ddots & \\ & & (1-\eta\lambda_n)^k \end{pmatrix} P^\top.$$

つまり $I_n - \eta A$ の固有値は $1 - \eta\lambda_i$ で，固有ベクトルは A と同じ p_i です．

Proof. $A = PDP^\top$ なので $A^k = PDP^\top PDP^\top \cdots = PD^kP^\top$．$I_n = PP^\top$ なので $I_n - \eta A = P(I_n - \eta D)P^\top$ となり，同じ計算ができます． □

解説

行列を k 回かけるのは大変ですが，対角行列なら**対角成分をそれぞれ k 乗するだけ**です．これは前回の「数列への利用」で使った $A^m = PD^mP^{-1}$ と同じ考え方です．

2.2 2 次形式と正定値行列

Def 1.9 (2 次形式, 正定値)**.** 対称行列 A について，$x^\top Ax = \sum_{i=1}^{n}\sum_{j=1}^{n} a_{ij}x_ix_j$ を ***2* 次形式**といいます．$x \neq \mathbf{0}$ ならいつでも $x^\top Ax > 0$ となるとき，A は**正定値**であるといいます．

解説

2 次形式は，2 変数なら $ax_1^2 + 2bx_1x_2 + cx_2^2$ のような「2 次の項だけの式」です．例えば $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ なら $x^\top Ax = 2x_1^2 + 2x_1x_2 + 2x_2^2$．正定値とは，グラフが**お椀の形**になる（原点からどの向きに進んでも値が上がる）ことです．

Thm 1.10. *Thm 1.4* の記号で $c = P^\top x$ とすると

$$x^\top Ax = \lambda_1 c_1^2 + \cdots + \lambda_n c_n^2.$$

特に，A が正定値であることと，A の固有値がすべて正であることは同じです．

Proof. $x^\top Ax = x^\top PDP^\top x = c^\top Dc = \sum \lambda_i c_i^2$．固有値がすべて正なら，$x \neq \mathbf{0}$ のとき $c \neq \mathbf{0}$ なので和は正です．逆に $x = p_i$ とすると $p_i^\top Ap_i = \lambda_i$ で，これが正になります． □

解説

$2x_1^2 + 2x_1x_2 + 2x_2^2$ は x_1x_2 の項があって形が分かりにくいですが，固有ベクトルの座標 $c_1 = \dfrac{x_1 - x_2}{\sqrt{2}}$, $c_2 = \dfrac{x_1 + x_2}{\sqrt{2}}$ で書き直すと $c_1^2 + 3c_2^2$ となり，固有値 1, 3 がそのまま係数に出てきます．係数がどちらも正なので，お椀の形だと一目で分かります．

Cor 1.11. A の固有値のうち最小のものを $\lambda_{\min}$，最大のものを $\lambda_{\max}$ とすると，$\lambda_{\min}\|x\|^2 \leq x^\top Ax \leq \lambda_{\max}\|x\|^2$ です．

Proof. Thm 1.10 の式で各 λ_i を $\lambda_{\min}$，$\lambda_{\max}$ に置きかえて比べ，Prop 1.7 の $c_1^2 + \cdots + c_n^2 = \|x\|^2$ を使います． □

2.3 勾配と 2 次関数

Def 1.12 (勾配)**.** $x \in \mathbb{R}^n$ の関数 f の偏微分を並べたベクトル

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}(x),\ \ldots,\ \frac{\partial f}{\partial x_n}(x)\right)^\top$$

を，f の**勾配**といいます．

解説

勾配は，山の斜面で**いちばん急な登り坂の向き**を指す矢印です．その逆向き $-\nabla f$ がいちばん急な下り坂なので，勾配降下法はその向きに進みます．実際，長さ 1 の向き u に進んだときの変化の割合は $\nabla f(x)^\top u$ で，これがいちばん小さくなるのは，u が $-\nabla f(x)$ と同じ向きのときです．

Prop 1.13. 定数ベクトル b と対称行列 A について，$\nabla(b^\top x) = b$，$\nabla(x^\top Ax) = 2Ax$ です．

Proof. $x^\top Ax$ のうち x_k を含む項は $a_{kk}x_k^2 + \sum_{j\neq k}(a_{kj} + a_{jk})x_kx_j$ で，$a_{kj} = a_{jk}$ なので，x_k で微分すると $2\sum_j a_{kj}x_j$ になります．1 つ目も同じように計算できます． □

Thm 1.14 (2 次関数の最小点)**.** A を正定値対称行列，$b \in \mathbb{R}^n$ として，$f(x) = \dfrac{1}{2}x^\top Ax - b^\top x$ とします．

1. $\nabla f(x) = Ax - b$.
2. f の最小点は *1* つだけで，$x^* = A^{-1}b$ です．さらに $f(x) - f(x^*) = \dfrac{1}{2}(x - x^*)^\top A(x - x^*)$.

Proof. 1 は Prop 1.13 から分かります．2: A は正定値なので固有値がすべて正で（Thm 1.10），$\det A \neq 0$ より逆行列 A^{-1} があります．$Ax^* = b$ を使って右辺を展開すると $\dfrac{1}{2}x^\top Ax - b^\top x + \dfrac{1}{2}x^{*\top}Ax^*$ になり，$f(x^*) = -\dfrac{1}{2}x^{*\top}Ax^*$ なので等式が成り立ちます．A は正定値なので右辺は 0 以上で，0 になるのは $x = x^*$ のときだけです． □

解説

1 変数の $f(x)=\dfrac{a}{2}x^2-bx$（$a>0$）なら，最小点は $x^*=b/a$ で，$f(x)-f(x^*)=\dfrac{a}{2}(x-x^*)^2$ です（平方完成）．Thm 1.14 は，これを行列とベクトルでそのまま書いたものです．正定値の A が「$a>0$」の役目をしています．

Ex 1.15. データ $(x,y)=(1,2),(2,3),(3,5),(4,6)$ に直線 $\hat{y}=w_1x+w_2$ を当てはめます．ここではデータの値を x,y，動かす数を $w=(w_1,w_2)^\top$ と書きます．損失関数

$$L(w)=\frac{1}{2\cdot 4}\sum_{i=1}^{4}(w_1x_i+w_2-y_i)^2$$

を $\dfrac{1}{2}w^\top Aw-b^\top w+(\text{定数})$ の形に書いて，最小点を求めてください．

SoL. $a_i=\begin{pmatrix}x_i\\1\end{pmatrix}$ とおくと $(w_1x_i+w_2-y_i)^2=w^\top a_ia_i^\top w-2y_ia_i^\top w+y_i^2$ なので，

$$A=\frac{1}{4}\sum_{i=1}^{4}a_ia_i^\top=\begin{pmatrix}15/2 & 5/2\\ 5/2 & 1\end{pmatrix},\qquad b=\frac{1}{4}\sum_{i=1}^{4}y_ia_i=\begin{pmatrix}47/4\\4\end{pmatrix}.$$

Thm 1.14 から，最小点は $Aw=b$ の解 $w=(7/5,\ 1/2)$ です．つまり直線 $\hat{y}=1.4x+0.5$ で，多変量解析で学んだ単回帰の公式の答えと一致します．

解説

これは，いちばん簡単な AI「直線の当てはめ（線形回帰）」の学習そのものです．w_1,w_2 が AI のパラメータ，$L(w)$ が損失関数（予測と正解の差の 2 乗の平均を，半分にしたもの）にあたります．データに 1 次式を当てはめる問題（最小 2 乗法）では，損失関数はいつも 2 次関数になります．

3　AI の学習への利用

ここからは AI の記号に合わせて，動かす数（パラメータ）を w，損失関数を $L(w)$ と書きます．

3.1　勾配降下法

L を小さくするには，L がいちばん速く減る向き $-\nabla L$ に少しずつ進めばよさそうです．

Def 1.16 (勾配降下法)**.** 初期値 w_0 と $\eta>0$ を決めて，

$$w_{k+1}=w_k-\eta\,\nabla L(w_k)\qquad(k=0,1,2,\ldots)$$

をくり返す方法を**勾配降下法**，η を**学習率**といいます．

まず 1 次元で見てみます．$L(w)=\dfrac{\lambda}{2}w^2$（$\lambda>0$，最小点は 0）なら $w_{k+1}=w_k-\eta\lambda w_k=(1-\eta\lambda)w_k$ なので，$w_k=(1-\eta\lambda)^kw_0$ です．λ（2 階微分 L''）が大きいほど，谷の両側の斜面が急になります．

$\eta\lambda$	公比 $1-\eta\lambda$	点の動き
$0<\eta\lambda<1$	0 と 1 の間	少しずつ近づく
$\eta\lambda=1$	0	1 回で着く
$1<\eta\lambda<2$	-1 と 0 の間	行き過ぎて，左右に揺れながら近づく
$\eta\lambda=2$	-1	同じ幅で揺れ続ける
$\eta\lambda>2$	-1 より小さい	揺れがどんどん大きくなり，飛び出す

解説

谷を下るボールを思い浮かべてください．歩幅が小さいと，少しずつ確実に下ります．歩幅が大きすぎると谷の反対側の斜面に飛び移り，もっと大きいと谷から飛び出してしまいます．谷が急なほど（λ が大きいほど），歩幅を小さくしなければなりません．

多次元でも同じことが起こるかを，対角化で調べます．

3.2 誤差の漸化式と対角化

ここからは，損失関数が Thm 1.14 の 2 次関数 $L(w)=\dfrac{1}{2}w^\top Aw-b^\top w$ （A は正定値対称行列）の場合を考えます．最小点を w^* とし，固有値を小さい順に $\lambda_{\min}=\lambda_1\le\cdots\le\lambda_n=\lambda_{\max}$ と並べます（固有ベクトル $p_1,\dots,p_n$ も同じ順に並べます）．

誤差を $e_k:=w_k-w^*$ とおきます．$Aw^*=b$ なので $\nabla L(w_k)=Aw_k-b=Ae_k$ となり，

$$e_{k+1}=e_k-\eta Ae_k=(I_n-\eta A)e_k,\qquad \text{つまり}\qquad e_k=(I_n-\eta A)^k e_0$$

です．これは前回の数列の $v_n=A^{n-k+1}v_{k-1}$ と同じ形をしています．Prop 1.7 で $e_0=c_1p_1+\cdots+c_np_n$ と展開して Prop 1.8 を使うと，

$$e_k=(1-\eta\lambda_1)^k c_1p_1+\cdots+(1-\eta\lambda_n)^k c_np_n,\qquad \|e_k\|^2=\sum_{i=1}^{n}(1-\eta\lambda_i)^{2k}c_i^2$$

となります．つまり，**固有ベクトルの方向ごとに，1 次元の問題が n 個並んでいるだけ**です．

解説

多次元の谷は，向きによって急さが違います．固有ベクトル p_i の向きが谷の「軸」で，固有値 λ_i がその向きの急さです．誤差をこの軸ごとの成分に分けると，各成分は 1 回ごとに $(1-\eta\lambda_i)$ 倍されるだけなので，3.1 節の 1 次元の表がそのまま使えます．ここで対角化（座標を回すこと）が効いています．

3.3 収束の条件

Thm 1.17 (収束の条件)**.** 学習率 $\eta>0$ の勾配降下法が，どんな初期値 w_0 から始めても w^* に近づく（$w_k\to w^*$）のは，

$$\eta<\frac{2}{\lambda_{\max}}$$

のとき，またそのときに限ります．

Proof. ($\eta < 2/\lambda_{\max}$ のとき) $0 < \eta\lambda_i \le \eta\lambda_{\max} < 2$ なので $|1-\eta\lambda_i| < 1$ です．よって各方向の成分 $(1-\eta\lambda_i)^k c_i$ は 0 に近づき，$\|e_k\| \to 0$ となります．

($\eta \ge 2/\lambda_{\max}$ のとき) $1-\eta\lambda_{\max} \le -1$ です．$w_0 = w^* + p_n$（p_n は $\lambda_{\max}$ の固有ベクトル）から始めると，$\|e_k\| = |1-\eta\lambda_{\max}|^k \ge 1$ なので，w^* には近づきません． □

解説

どの軸の方向でも飛び出さないためには，**いちばん急な方向**（固有値が最大の $\lambda_{\max}$ の方向）で $\eta\lambda_{\max} < 2$ であればよい，ということです．学習率の上限は，谷のいちばん急な方向で決まります．

Ex 1.18. $A = \begin{pmatrix} 11/2 & 9/2 \\ 9/2 & 11/2 \end{pmatrix}$ のとき，勾配降下法が収束する学習率の範囲を求めてください．

SoL. 固有多項式は $(t-\frac{11}{2})^2 - (\frac{9}{2})^2 = (t-1)(t-10)$ なので，固有値は 1, 10 です．*Thm 1.17* から，収束するのは $0 < \eta < 2/10 = 0.2$ のときです．例えば $\eta = 0.25$ にすると，固有値 10 の固有ベクトル $\frac{1}{\sqrt{2}}(1,1)^\top$ の方向の誤差が *1* 回ごとに $|1-2.5| = 1.5$ 倍になり，発散します．

注意. 「どんな初期値から始めても」という条件は外せません．*Ex 1.18* で $\eta = 0.3$ にしても，誤差 e_0 が $(1,-1)^\top$ の方向なら，分数で正確に計算すると $e_{60} = 0.7^{60}(1,-1)^\top$ となり，収束します．ところが，例えば $w^* = (1,2)^\top$，$w_0 = w^* + (1,-1)^\top$ として，w_k をコンピュータの小数（浮動小数点）で計算すると，丸め誤差で $(1,1)^\top$ 方向の成分がわずかに生まれます．それが *1* 回ごとに $|1-0.3\cdot 10| = 2$ 倍になり，120 回後には誤差が 10^{18} ほどまで大きくなりました．

3.4 最適な学習率と条件数

収束する範囲の中で，どの η がいちばん速いかを考えます．1 回あたりにどれだけ縮むかは

$$\rho(\eta) := \max_i |1-\eta\lambda_i| = \max\left(|1-\eta\lambda_{\min}|,\ |1-\eta\lambda_{\max}|\right)$$

で決まります．$1-\eta\lambda$ は λ が大きいほど小さくなるので，いちばん小さい固有値といちばん大きい固有値だけを見れば十分です．

Thm 1.19 (最適な学習率)**.** $\|e_{k+1}\| \le \rho(\eta)\|e_k\|$ です．$\rho(\eta)$ をいちばん小さくする学習率と，そのときの値は

$$\eta^* = \frac{2}{\lambda_{\min}+\lambda_{\max}}, \qquad \rho(\eta^*) = \frac{\kappa-1}{\kappa+1} \qquad \left(\kappa := \frac{\lambda_{\max}}{\lambda_{\min}}\right)$$

です．κ を**条件数**といいます．

Proof. $\|e_{k+1}\|^2 = \sum(1-\eta\lambda_i)^2 c_i^2 \le \rho^2 \sum c_i^2 = \rho^2\|e_k\|^2$（$c_i$ は e_k を展開したときの係数）．次に，$1-\eta\lambda_{\min} = \eta\lambda_{\max} - 1$ を解くと $\eta = \eta^*$ です．$0 < \eta \le \eta^*$ では，2 つのうち大きい方は $1-\eta\lambda_{\min}$ で，η が大きくなると減ります．$\eta \ge \eta^*$ では，大きい方は $\eta\lambda_{\max} - 1$ で，η が大きくなると増えます．よって $\rho(\eta)$ は $\eta = \eta^*$ でいちばん小さくなり，$\rho(\eta^*) = 1-\eta^*\lambda_{\min} = \frac{\lambda_{\max}-\lambda_{\min}}{\lambda_{\max}+\lambda_{\min}} = \frac{\kappa-1}{\kappa+1}$ です． □

解説

条件数は「谷の細長さ」です．Thm 1.14 と Thm 1.10 から，損失の等高線 $L(w)-L(w^*)=1$ は，p_i の方向の半径が $\sqrt{2/\lambda_i}$ の楕円（3 次元以上なら楕円体）になり，いちばん長い軸といちばん短い軸の長さの比は $\sqrt{\kappa}$ です．細長い谷では，急な方向に合わせて歩幅を小さくするしかないので，緩やかな方向にはなかなか進めず，谷を行ったり来たりしながらゆっくり進みます．$\kappa=1$（まん丸の谷）なら，$\eta=\eta^*$ のとき $\rho=0$ で，1 回で谷底に着きます．

条件数 κ が大きいほど $\rho(\eta^*)$ は 1 に近づき，遅くなります．Thm 1.17 と Thm 1.19 は，解説論文 [4] にも書かれています．また，最適化の教科書 [3] の 9.3 節でも，学習率の決め方は違いますが，勾配降下法の速さが条件数で決まることが説明されています．では，何回くり返せば十分でしょうか．以下，log は自然対数です．

Lem 1.20. $\kappa>1$ なら $\log\dfrac{\kappa+1}{\kappa-1}\geq\dfrac{2}{\kappa}$.

Proof. $t=1/\kappa\in(0,1)$，$g(t)=\log\dfrac{1+t}{1-t}-2t$ とおくと，$g(0)=0$ で，$g'(t)=\dfrac{1}{1+t}+\dfrac{1}{1-t}-2=\dfrac{2}{1-t^2}-2\geq 0$．よって $g(t)\geq 0$ です． □

Cor 1.21. $\kappa>1$，$0<\varepsilon<1$ とします．$\eta=\eta^*$ のとき，$N\geq\dfrac{\kappa}{2}\log\dfrac{1}{\varepsilon}$ 回くり返せば，誤差は ε 倍以下になります．

Proof. Thm 1.19 から $\|e_N\|\leq\rho^N\|e_0\|$ です．$\rho^N\leq\varepsilon$ は $N\log\dfrac{1}{\rho}\geq\log\dfrac{1}{\varepsilon}$ と同じで，Lem 1.20 から $\log\dfrac{1}{\rho}\geq\dfrac{2}{\kappa}$ なので，$N\geq\dfrac{\kappa}{2}\log\dfrac{1}{\varepsilon}$ なら十分です． □

つまり，**必要な回数は条件数にほぼ比例します**．実際に $A=\mathrm{diag}(1,\kappa)$，$e_0=(1,1)^\top$ として，誤差が 10^{-6} 倍になるまでの回数を数えると，次のようになりました．

条件数 κ	10	100	1000
実際の回数	69	691	6908
$\dfrac{\kappa}{2}\log 10^6$	69.1	690.8	6907.8

問. A の固有値が 2 と 8 のとき，収束する学習率の範囲，最適な学習率，そのときの *1* 回あたりの縮み方を求めてください．

SoL. *Thm 1.17* から $0<\eta<2/8=0.25$．*Thm 1.19* から $\eta^*=2/(2+8)=0.2$ で，$\kappa=4$ なので $\rho=3/5=0.6$ です．

3.5 例：直線の当てはめ

Ex 1.15 の直線の当てはめを，勾配降下法で学習させてみます．

Ex 1.22. *Ex 1.15* の損失関数で，勾配降下法が収束する学習率の範囲と，条件数を求めてください．

SoL. $A=\begin{pmatrix}15/2 & 5/2\\ 5/2 & 1\end{pmatrix}$ の固有多項式は $t^2-\dfrac{17}{2}t+\dfrac{5}{4}$ なので，固有値は $\dfrac{17\pm\sqrt{269}}{4}$，つまり約 8.350 と

約 0.1497 です．よって *Thm 1.17* から，収束するのは $0 < \eta < 2/8.350 \approx 0.2395$ のときで，条件数は $\kappa \approx 55.8$ です．実際に計算すると，$\eta = 0.239$ なら収束し，$\eta = 0.240$ なら発散しました．また最適な学習率でも，$w_0 = (0, 0)$ から誤差が 10^{-6} 未満になるまで 397 回かかりました．

データがたった 4 個でも，条件数が大きいと学習は遅くなります（等高線の楕円は，縦横比がおよそ 7.5 倍の細長さです）．そこで，x から平均 2.5 を引いてから当てはめる（**中心化**する）と，$\hat{y} = w_1(x - 2.5) + w_2$ の損失関数の行列は

$$A = \begin{pmatrix} 5/4 & 0 \\ 0 & 1 \end{pmatrix}, \qquad \kappa = 1.25$$

となり，最適な学習率で同じ精度になるまで **7 回**で済みました．

解説

x の値がすべて正だと，「傾き w_1 を増やす」と「切片 w_2 を増やす」がどちらも予測を上げるので，2 つのパラメータの働きが重なります．これが A の非対角成分 $5/2$ として現れ，谷が斜めに細長くなっていました．中心化でこの重なりがなくなり，谷がまん丸に近づいたのです．多変量解析で学ぶ標準化（平均 0，分散 1 にそろえること）も，変数の尺度をそろえて条件数を小さくする方法です．説明変数が 2 つの回帰では，標準化が「各変数を何倍かしてそろえる方法」の中で条件数をいちばん小さくすることも示せます（説明変数が 3 つ以上だと，いつもそうなるとは限りません）．

3.6 実際の AI のどこで使われているか

ここまでの結果が，実際の AI のどこに出てくるかをまとめます．

■(1) AI の学習のプログラム　AI を作るときによく使われる Python のライブラリ PyTorch では，学習を次のようなくり返しで書きます [8]．

プログラムの 1 行	意味
`optimizer.zero_grad()`	前回の勾配を 0 に戻す
`output = model(input)`	AI に入力を入れて予測を計算する
`loss = loss_fn(output, target)`	損失関数 L を計算する
`loss.backward()`	勾配 ∇L を計算する
`optimizer.step()`	パラメータを更新する

更新の方法に `torch.optim.SGD` を選び，`momentum` などを指定しなければ，最後の `optimizer.step()` が勾配降下法の 1 回 $w_{k+1} = w_k - \eta \nabla L(w_k)$ になります．このとき設定する `lr`（learning rate，既定値 0.001）が学習率 η です [9]．損失が 2 次関数なら，**Thm 1.17 が「`lr` をどこまで大きくしてよいか」の答え**になります．

■(2) 回帰（数値を予測する AI）　3.5 節の直線の当てはめのように，データに 1 次式を当てはめる AI（線形回帰）では，損失関数がちょうど 2 次関数です．だから Thm 1.17，Thm 1.19 がそのまま成り立ちます．学習の前に入力を中心化・標準化すると速くなることがあるのも，条件数で説明できます．

■(3) **ニューラルネットワーク**　画像や文章を扱う AI（ニューラルネットワーク）の損失関数は，2 次関数ではありません．それでも損失がいちばん小さい点の近くでは 2 次関数で近似でき，そのとき A にあたるのが，2 階偏微分を並べた行列（**ヘッセ行列**）です．1 次元の場合は，最小点での 2 階微分を λ として，$0 < \eta\lambda < 2$ なら最小点の十分近くから収束し，$\eta\lambda > 2$ ならその最小点には収束できないことが，平均値の定理を使って示せます．つまり Thm 1.17 は，「学習率 η の勾配降下法が，その谷底に落ち着けるかどうか」の条件として出てきます．

■(4) **実際の学習で見つかった現象**　実際のニューラルネットワークを全データの勾配降下法で学習すると，ヘッセ行列の最大固有値が，$2/\eta$ のすぐ上にとどまることが報告されています（Edge of Stability）[5]．Thm 1.17 の境目 $\eta\lambda_{\max} = 2$ が，本物の AI の学習にも現れているのです．ただし，2 次関数なら発散するはずの状態なのに，実際には損失が長い目で見ると下がっていきます．なぜそうなるかは，2 次関数の理論だけでは説明できません（4 節）．

■(5) **勾配降下法を改良した方法**　実際の学習では，次のような改良版もよく使われます．

- **ミニバッチ（確率的勾配降下法）**：毎回データの一部だけで勾配を計算して，1 回を軽くする方法です．学習率を一定にすると，最小値の近く（学習率に比例する幅）までしか近づけないことが示されています [6]．
- **モーメンタム**：前回動いた分の β 倍だけ，さらに進む方法です（PyTorch の `momentum`）．2 次関数の場合，$0 \le \beta < 1$ なら，収束する学習率の上限が $2(1+\beta)/\lambda_{\max}$ に広がります [4]．
- **Adam**：勾配の大きさで割って，パラメータごとに歩幅を調整する方法です [7]．PyTorch では `torch.optim.Adam` という名前で入っています．

どれも「勾配を使って少しずつ動く」という勾配降下法の考え方が土台になっています．

4　まとめ

今回は，AI の学習の基本である勾配降下法を，実対称行列の対角化を使って調べました．損失関数が 2 次関数のとき，次のことが分かりました．

1. どんな初期値からでも収束するのは，$0 < \eta < 2/\lambda_{\max}$ のときで，そのときに限る（Thm 1.17）．
2. 最適な学習率は $\eta^* = \dfrac{2}{\lambda_{\min} + \lambda_{\max}}$ で，必要な回数は条件数 κ にほぼ比例する（Thm 1.19，Cor 1.21）．
3. 条件数が小さいほど速い．中心化や標準化で条件数を小さくできることがある．

言いかえると，「歩幅の上限は谷のいちばん急な方向で決まり，速さは谷の細長さで決まる」ということです．前回の数列と同じく，「行列の累乗を対角化で計算する」ことが鍵でした．そしてこの結果は，AI のプログラムの学習率 `lr` の決め方や，実際の AI の学習で見つかった現象（Edge of Stability）にもつながっています．

一方で，まだ分かっていないことや，これから調べたいこともあります．

- 変数を 1 つずつ交互に更新する方法は変数が 2 つの場合，学習率を交互に使う方法は学習率が 2 つの場合について，結果が得られています．これを変数や学習率が 3 つ以上の場合に広げられるか．
- Edge of Stability（3.6 節 (4)）で，最大固有値が $2/\eta$ を超えていても損失が下がっていくのはなぜか．
- AI でよく使われる Adam のように，勾配の大きさで割って歩幅を調整する方法では，どんな学習率な

ら収束するか.

次は，このあたりを勉強したいと考えています.

参考文献

[1] 田森 宥好, 線形代数 *2* ノート, （参照 2026-05-15）.

[2] 多変量解析 講義資料「線形代数のまとめ」「対称行列について」, 2026 年度後期, （参照 2026-10-30）.

[3] S. Boyd, L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004, §9.3, `https://web.stanford.edu/~boyd/cvxbook/`, （参照 2026-10-31）.

[4] G. Goh, *Why Momentum Really Works*, Distill, 2017, `https://distill.pub/2017/momentum/`, （参照 2026-10-31）.

[5] J. M. Cohen, S. Kaur, Y. Li, J. Z. Kolter, A. Talwalkar, *Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability*, ICLR 2021, `https://arxiv.org/abs/2103.00065`, （参照 2026-10-31）.

[6] L. Bottou, F. E. Curtis, J. Nocedal, *Optimization Methods for Large-Scale Machine Learning*, SIAM Review 60(2), 2018, `https://arxiv.org/abs/1606.04838`, （参照 2026-10-31）.

[7] D. P. Kingma, J. Ba, *Adam: A Method for Stochastic Optimization*, 2014, `https://arxiv.org/abs/1412.6980`, （参照 2026-10-29）.

[8] PyTorch documentation, *torch.optim*, `https://docs.pytorch.org/docs/2.15/optim.html`, （参照 2026-10-31）.

[9] PyTorch documentation, *torch.optim.SGD*, `https://docs.pytorch.org/docs/2.15/generated/torch.optim.SGD.html`, （参照 2026-10-29）.